Software Architecture for Essential & Accidental Uncertainty

Neil Harrison, **George Rudolph** (presenter), **Peter Aldous**
Utah Valley University
Orem, Utah

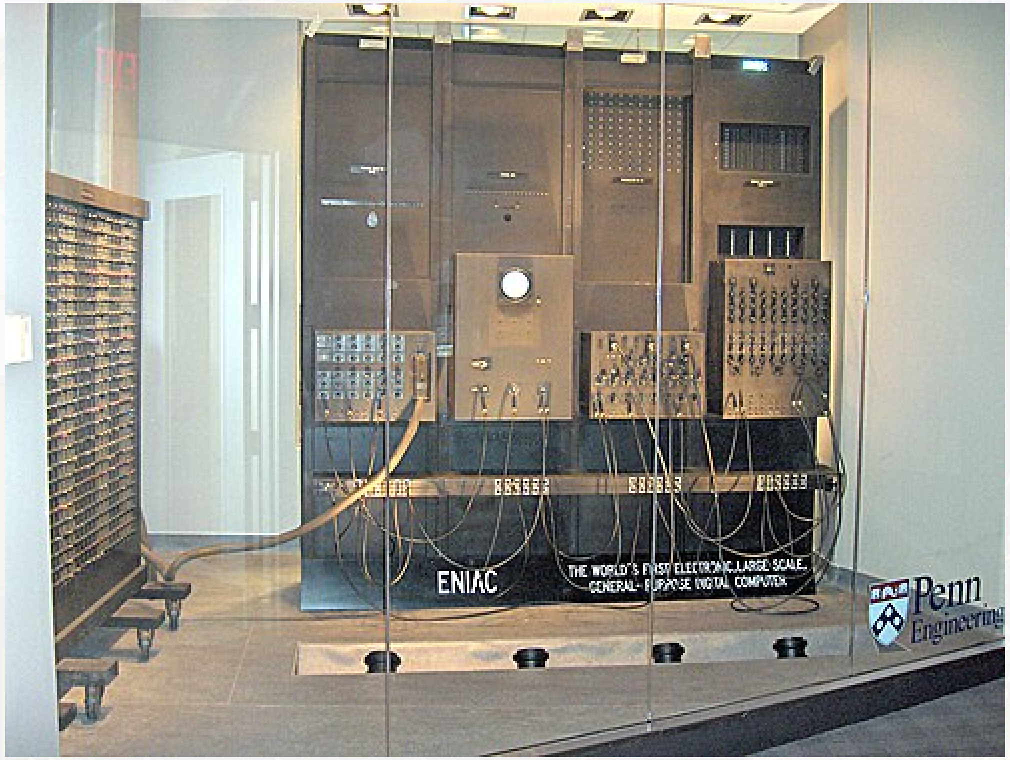
The Challenge: Uncertainty in Modern Computing

Early computing: Precisely specified problems with deterministic outputs

Today: Computing expanded to business, AI, control systems

- Inputs, outputs, and processes became more varied and less precise
- We must deal with uncertainty in architecture and design

Current practice focuses on fault tolerance, but **this misses a large part of uncertainty**.



Essential vs. Accidental Uncertainty

Accidental uncertainty: can be separated from the problem; *a transformation exists* to remove it.

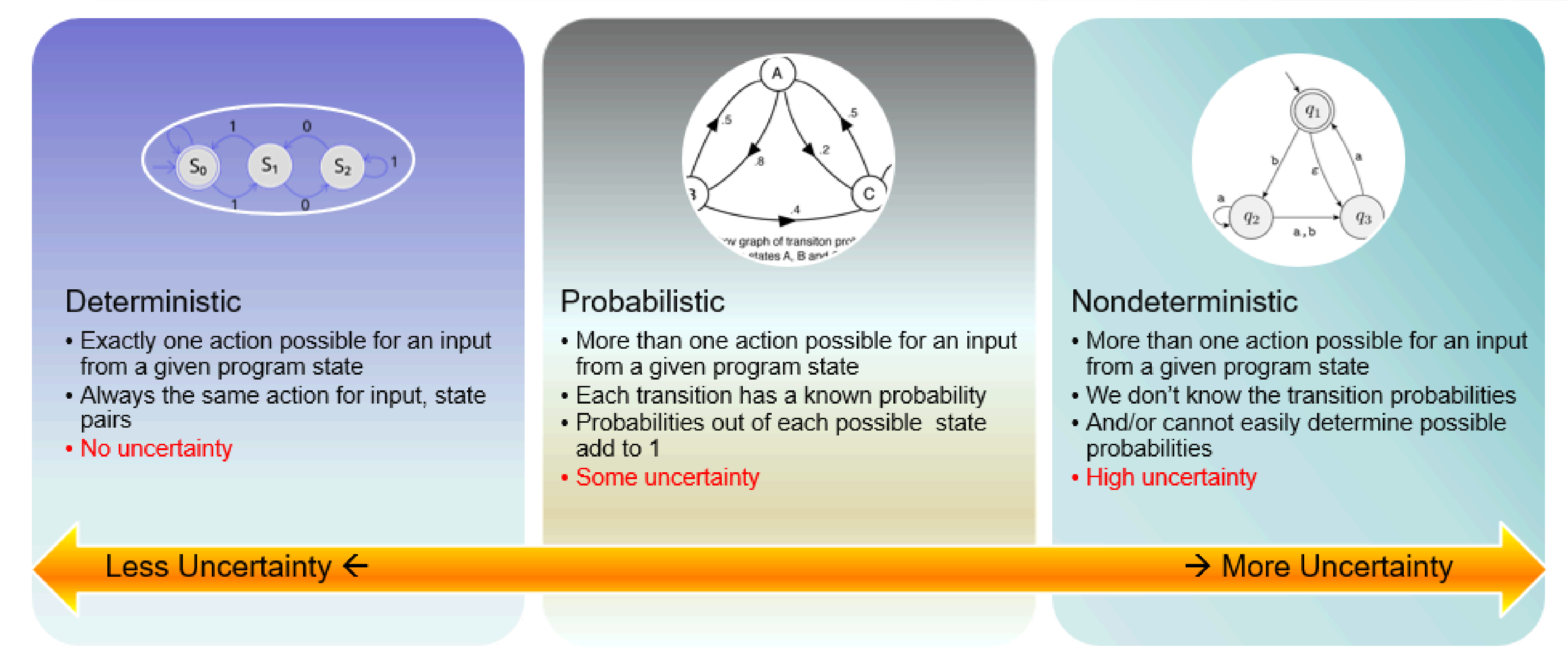
Essential uncertainty: must be addressed as part of the problem; *no transformation* can remove it.

Framework: Consider uncertainty across **Input**, **Transformation**, and **Output**.

Essential Output Uncertainties

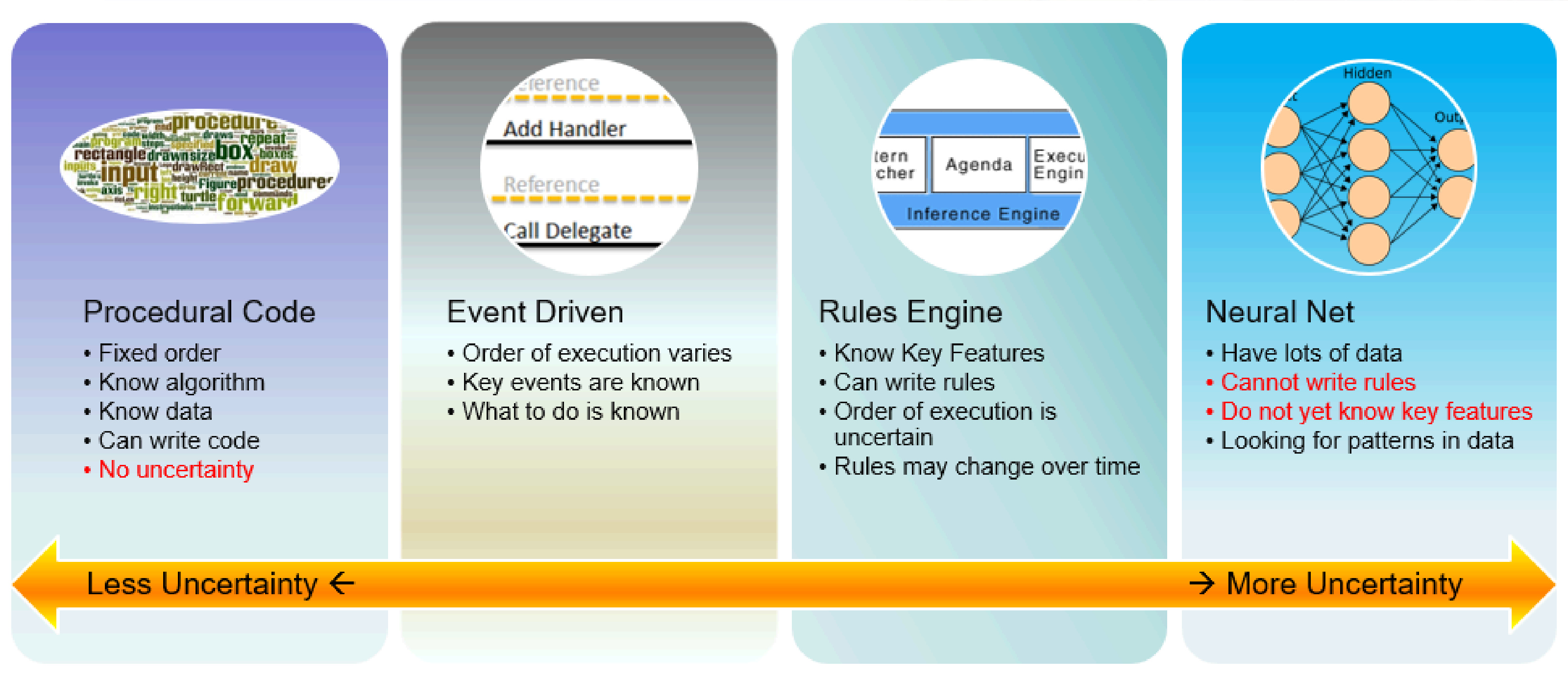
Output characteristics	Example
There is a known right answer	Mathematical computation
The right answer(s) can be known	Sales website (How well did a product sell?)
Right answer exists but is infeasible to compute directly	Game of Go (astronomical state space)
Unknown right answer; we can **guess**	Image recognition
Effect is unknown, but can be measured	Optimization problems
Effect is unknown; system is **chaotic**	Weather forecasting
No specific answer, but can observe "right" direction	Advertising; many simulation problems
No single right answer; the problem is **vague**	Automated prose writing
The problem itself is unknown	Grounded theory

Essential Output Uncertainties Spectrum



Range: From known right answers (mathematical computation) to unknown problems (grounded theory)

Architectural Approaches



Match architecture to uncertainty: Batch processing → Pipes & Filters → AI-based approaches

Architecture Process for Uncertainty

1. **Identify** uncertainties (Input, Transformation, Output)
2. Determine if **essential** or **accidental**
3. Can a **transformation** remove it?
   - **Yes** → Apply fault-tolerance techniques
   - **No** → Use AI-based approaches

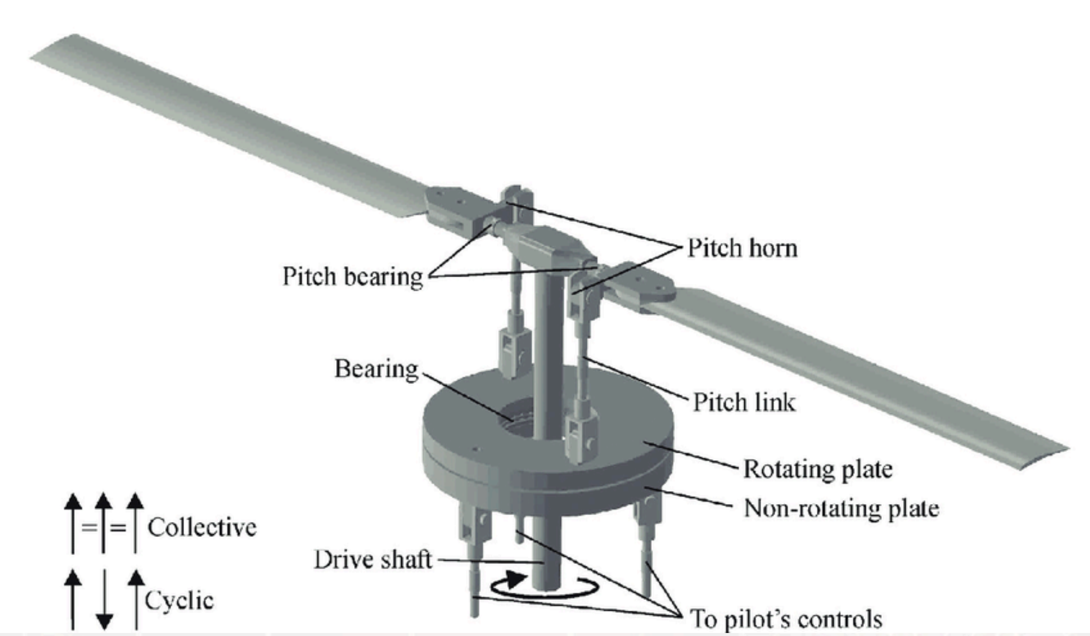
Case Study: Helicopter Rotor Balancing

Problem: Unbalanced rotor causes vibration, wear, loss of power, safety issues

Uncertainty Analysis:

- **Input**: Known measurements; unique wear patterns (essential)
- **Output**: Measurable (IPS); multiple solution sequences possible
- **Processing**: Initially appeared non-deterministic/chaotic

Initial approach: Neural network (treating as essential uncertainty)



Case Study: Helicopter Rotor Balancing (continued)

Further analysis revealed:

- Each helicopter has a **unique wear profile**
- Initial NN trained on many helicopters → generic approximation
- **Adjustment/response history** reveals unique profiles
- **Key insight**: There exists a **deterministic, repeatable** sequence of adjustments with **feedback loops**

What appeared essential was actually accidental, solvable with the right approach.

Results: System balances rotors to **0.05 IPS** — *10× better* than the 0.5 IPS standard. Actively marketed to US Army.

Case Study: Online Bookstore Evolution

Initially: Inputs/outputs well-specified; numerous **accidental uncertainties** (e.g., unreliable networks). Straightforward architecture.

Evolution 1 — Related Books: Which books are related? An **essential uncertainty** — output cannot be known beforehand, no single right answer. Approaches become **probabilistic** (recommendation algorithms).

Evolution 2 — Promotions: Using browsing history introduces even more uncertainty. Requires **machine learning** approaches, mapping to different positions on the uncertainty spectrum.

References

- Harrison, N.B., Rudolph, G., and Aldous, P. *Software Architecture for Essential and Accidental Uncertainty*. HICSS, 2026.
- Harrison, N.B., Rudolph, G., and Aldous, P. *Is Artificial Intelligence the Answer to Everything?* Intermountain Conference on Engineering, Technology and Computing (IETC), 2025.

Thank you!

Summary: We've presented a framework for understanding essential versus accidental uncertainty in software architecture. This distinction is crucial for selecting appropriate architectural approaches. Essential uncertainty requires probabilistic and AI-based methods, while accidental uncertainty can be addressed with traditional fault-tolerance techniques.

Questions?